

MoonBit 开源生态项目贡献赛 · 项目申报书

一、项目基本信息

项目	内容
项目名称	MoonBit Depsight — 依赖健康诊断器
申报人	[肖若愚]
团队名称	鱼仔爱开发
仓库地址	https://www.gitlink.org.cn/LittleFish/moonmark
主要实现语言	MoonBit
目标平台	mooncakes.io
项目类型	生态工具（CLI + 可视化报告）

二、项目背景与问题陈述

MoonBit 生态正处于高速扩张期。截至 2026 年，mooncakes.io 已收录超过 9000 个包，开发者在享受丰富依赖的同时，也面临以下痛点：

1. 依赖黑洞：引入一个看似轻量的包，实际可能附带数十个传递依赖，导致最终产物体积不可控。
 2. 废弃 API 蔓延：官方已支持 @deprecated 注解，但缺乏工具跨包检测"你的依赖是否在调用已废弃的接口"。
 3. 许可证风险：不同包采用 MIT、Apache-2.0、GPL 等协议，手动排查冲突成本极高。
 4. 版本漂移：moon.mod.json 里的版本约束缺乏可视化工具，开发者难以判断升级路径是否安全。
- 官方 moon CLI 目前仅提供 add / remove / update 等基础依赖管理能力，未覆盖依赖审计、健康评分、风险可视化等上层需求。社区在这一领域完全空白。

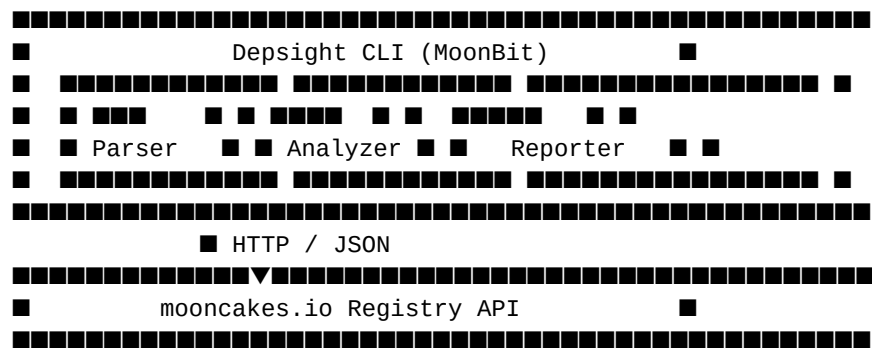
三、项目目标

构建一个以 MoonBit 为核心实现的依赖健康诊断工具，为 MoonBit 开发者提供以下能力：

- 依赖树解析：读取 moon.mod.json，递归拉取 mooncakes.io 元数据，构建完整的传递依赖图。
- 体积分析：统计每个传递依赖对最终构建产物的体积贡献，定位"体积罪魁祸首"。
- 废弃 API 检测：扫描依赖包源码或接口声明，标记已废弃的公开 API 及其被引用路径。
- 许可证合规：自动识别依赖树中各节点的开源协议，标记 GPL 等强 copyleft 冲突。
- 健康评分：基于上述维度，为每个依赖包计算综合健康评分，并生成可视化报告。

四、技术方案

4.1 架构设计



4.2 核心模块

模块	实现语言	职责
moon.mod.json 解析器	MoonBit	读取本地依赖声明，处理版本约束语法
元数据获取器	MoonBit	并发拉取 mooncakes.io 上各包的 moon.mod.json 与接口声明
依赖图构建器	MoonBit	构建有向无环图（DAG），检测循环依赖
废弃 API 扫描器	MoonBit	基于接口文件（.mi）或源码 AST，定位 @deprecated 标记
许可证识别器	MoonBit	解析 LICENSE 文件头，匹配 SPDX 标准协议标识符
报告渲染器	MoonBit → JS/WASM	生成终端表格（TTY）与 HTML 可视化报告

4.3 关键技术选型

- MoonBit 标准库：@moonbitlang/core 提供 JSON 解析、字符串处理、集合操作，无需引入额外依赖。
- 编译目标：CLI 主体编译为 WASM/JS，通过 Node.js 运行；核心算法模块编译为 WASM-GC 以追求性能。
- HTTP 客户端：使用 MoonBit JS FFI 调用 fetch 获取 mooncakes.io API 数据。
- 输出格式：支持终端富文本（类似 npm audit）、静态 HTML 报告、JSON（供 CI/CD 消费）。

五、创新点

1. 跨包废弃 API 传递分析

不仅检测直接依赖的废弃接口，更追踪"你的依赖所依赖的包"中是否存在已废弃的跨层调用，这是现有任何 MoonBit 工具都不具备的能力。API

2. 语义化体积归因

不是简单统计源码体积，而是结合 MoonBit 的编译特性，估算各传递依赖对最终产物的体积贡献（基于符号引用分析）。.wasm

3. 依赖健康评分模型

设计一套可量化的评分算法（0-100 分），综合考量版本新鲜度、协议兼容性、废弃 API 密度、维护活跃度，让"健康度"一目了然。

4. 原生 MoonBit 实现

核心解析器、图算法、分析引擎全部使用 MoonBit 编写，充分展示语言在系统工具开发上的表达能力，并反哺标准库的实战验证。

附录

- 比赛名称：MoonBit × CCF 开源生态项目贡献赛
- 技术栈：MoonBit / WASM / JavaScript FFI / HTML/CSS
- 开源协议：Apache-2.0
- 依赖声明：除 @moonbitlang/core 外，不引入第三方 MoonBit 依赖；构建时工具链使用 Node.js 18+。